

Шпаргалка по Machine Learning (ML)

9 июня 2026 г.

Основные этапы решения задачи машинного обучения:

1. Постановка задачи;
2. Выделение признаков;
3. Формирование выборки;
4. Выбор функционала ошибки;
5. Предобработка данных;
6. Построение модели;
7. Оценивание качества модели.

Содержание

1	Измерение ошибки в задачах регрессии	4
1.1	MSE (Mean Squared Error) и R^2	4
1.2	MAE (Mean Absolute Error)	4
1.3	Гибридные функции: Huber Loss и Log-Cosh	4
1.4	MSLE (Mean Squared Logarithmic Error)	5
1.5	Относительные метрики: MAPE и SMAPE	5
1.6	Квантильная функция потерь (Quantile Loss)	5
2	Градиентные методы оптимизации	6
2.1	Градиент и его свойства	6
2.2	Классический градиентный спуск (Gradient Descent, GD)	6
2.3	Оценивание градиента на больших данных	6
2.4	Модификации градиентного спуска	6
3	Линейная регрессия, переобучение и валидация	8
3.1	Обучение линейной регрессии (Метод наименьших квадратов)	8
3.2	Переобучение и Регуляризация	8
3.3	Оценивание качества моделей	8
4	Линейная классификация и метрики качества	9
4.1	Линейный классификатор	9
4.2	Отступ (Margin)	9
4.3	Метрики качества классификации	9
5	Логистическая регрессия и Метод опорных векторов	10
5.1	Логистическая регрессия (Logistic Regression)	10
5.2	Метод опорных векторов (Support Vector Machine — SVM)	10
6	Многоклассовая классификация	11
6.1	Сведение к серии бинарных задач	11
6.2	Многоклассовая логистическая регрессия	11
7	Классификация с пересекающимися классами (Multi-label)	12
7.1	Независимая классификация (Binary Relevance)	12
7.2	Стекинг классификаторов	12
7.3	Хэммингово расстояние (Метрика ошибки)	12

8	Обработка категориальных признаков	13
8.1	Бинарное кодирование (One-Hot Encoding / OHE)	13
8.2	Бинарное кодирование с хэшированием (Hashing Trick)	13
8.3	Счётчики (Target Encoding)	13
9	Решающие деревья: Базовые понятия	14
9.1	Математическая модель	14
9.2	Жадное построение (ID3 / C4.5 / CART)	14
9.3	Критерии останова (Регуляризация)	14
10	Критерии информативности $H(X)$	15
10.1	Задачи классификации	15
10.2	Задачи регрессии	15
11	Продвинутые аспекты решающих деревьев	16
11.1	Стрижка деревьев (Pruning)	16
11.2	Обработка пропущенных значений (Missing Values)	16
11.3	Связь решающих деревьев с линейными моделями	16
12	Разложение ошибки на смещение и разброс	17
12.1	Теоретическое обоснование	17
12.2	Математические компоненты разложения	17
12.3	Trade-off простых и сложных моделей	17
13	Бэггинг (Bootstrap Aggregation)	18
13.1	Бутстрап-выборки	18
13.2	Построение композиции бэггинга	18
13.3	Влияние бэггинга на Смещение и Разброс	18
14	Случайный лес (Random Forest)	19
14.1	Алгоритм построения	19
14.2	Рекомендуемые гиперпараметры (CART/Breiman)	19
14.3	Out-of-Bag (OOB) Валидация	19
14.4	Геометрическая интерпретация	19
15	Градиентный бустинг: Математические основы	20
15.1	Концепция последовательного обучения	20
15.2	Псевдоградиентный спуск в пространстве функций	20
15.3	Построение базового алгоритма	20
16	Регуляризация и Функции потерь бустинга	21
16.1	Методы борьбы с переобучением	21
16.2	Стандартные спецификации антиградиента	21
17	Градиентный бустинг над решающими деревьями (GBDT)	22
17.1	Особенности композиции над деревьями	22
17.2	Разделение задачи на подзадачи в листьях	22
17.3	Аналитические решения для листьев	22
18	Extreme Gradient Boosting (XGBoost)	23
18.1	Математическая суть и аппроксимация 2-го порядка	23
18.2	Регуляризация дерева в XGBoost	23
18.3	Построение дерева и критерии	23
19	Композиции: Стекинг и Блендинг	24
19.1	Стекинг (Stacking)	24
19.2	Блендинг (Blending)	24
19.3	Обработка категориальных и текстовых признаков через Стекинг	24
20	Ядровые методы: Теория и Трюк	25
20.1	Переход в спрямляющее пространство	25
20.2	Ядровой трюк (Kernel Trick)	25
20.3	Теорема Мерсера (Условие существования ядра)	25
20.4	Конструктор ядер (Свойства замыкания)	25
20.5	Основные типы ядер	25

21 Ядровой метод опорных векторов (Kernel SVM)	26
21.1 Двойственная задача линейного SVM	26
21.2 Применение ядрового трюка к SVM	26
21.3 Классификация новых объектов	26
21.4 Плюсы и минусы ядрового SVM	26
21.5 Метод случайных признаков (Random Fourier Features)	26
22 Смеси распределений и ЕМ-алгоритм	27
22.1 Смеси распределений	27
22.2 Проблема неполных данных и сломанный ММП	27
22.3 Нижняя вариационная оценка правдоподобия (ELBO)	27
22.4 Итерации ЕМ-алгоритма	27
22.5 Гауссова смесь (GMM — Gaussian Mixture Model)	27
22.6 Свойства ЕМ-алгоритма	28
23 Несбалансированные выборки и Поиск аномалий	29
23.1 Методы сэмплирования выборки	29
23.2 Одноклассовая классификация (One-Class Classification)	29
23.3 One-Class SVM (OC-SVM)	29
23.4 Изоляционный лес (Isolation Forest)	29
24 Графовая и спектральная кластеризация	31
24.1 Представление выборки в виде графа	31
24.2 Критерии разреза графа	31
24.3 Матрица Лапласа графа (Laplacian)	31
24.4 Алгоритм спектральной кластеризации (по Ши и Малику)	31
24.5 Внешние метрики качества кластеризации	32
25 Метрические методы классификации и LSH	33
25.1 Классификатор к ближайших соседей (kNN)	33
25.2 Проклятие размерности в метрических пространствах	33
25.3 Приближенный поиск соседей (ANN) и LSH	33
25.4 LSH для евклидова расстояния (Метод случайных проекций)	33
25.5 Увеличение точности: Конструкции AND и OR	34

1 Измерение ошибки в задачах регрессии

Обозначения: y — истинный ответ, a — прогноз модели, ℓ — размер выборки.

1.1 MSE (Mean Squared Error) и R^2

Формула MSE:

$$\text{MSE}(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} (a(x_i) - y_i)^2$$

Формула RMSE:

$$\text{RMSE}(a, X) = \sqrt{\text{MSE}(a, X)}$$

Коэффициент детерминации (R^2):

$$R^2(a, X) = 1 - \frac{\sum_{i=1}^{\ell} (a(x_i) - y_i)^2}{\sum_{i=1}^{\ell} (y_i - \bar{y})^2}, \quad \text{где } \bar{y} = \frac{1}{\ell} \sum_{i=1}^{\ell} y_i$$

- **Применяется:** Как базовый функционал ошибки во время обучения. Благодаря гладкости и дифференцируемости функции $L(y, a) = (a - y)^2$, оптимизация (например, градиентный спуск) проходит быстро и стабильно. Минимум достигается на **среднем значении** ответов.
- **Не применяется:** При наличии сильных аномалий и шумов. Из-за возведения ошибки в квадрат MSE нелинейно штрафует за крупные промахи, заставляя модель подстраиваться под выбросы в ущерб нормальным данным. Ошибка плохо интерпретируется (измеряется в квадратах единиц).

1.2 MAE (Mean Absolute Error)

Формула MAE:

$$\text{MAE}(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} |a(x_i) - y_i|$$

- **Применяется:** В задачах, где данные содержат редкие **выбросы** (аномальные значения), так как штраф растет линейно и модель к ним устойчива. Оптимальный ответ сводится к **медиане** ответов. Ошибка легко интерпретируется человеком (сохраняет исходные единицы измерения).
- **Не применяется:** Если важна быстрая и простая сходимость алгоритма. Производная модуля равна $\text{sign}(a - y)$ и не зависит от близости к оптимуму, из-за чего обучение идет дольше, а алгоритм может «перескакивать» экстремум. Особенность устойчивости пропадает, если выбросов становится слишком много (около половины выборки).

Сравнение поведения MSE и MAE при выбросах

y	$a_1(x)$	$(a_1(x) - y)^2$	$ a_1(x) - y $	$a_2(x)$	$(a_2(x) - y)^2$	$ a_2(x) - y $
1	2	1	1	4	9	3
2	1	1	1	5	9	3
3	2	1	1	6	9	3
4	5	1	1	7	9	3
5	6	1	1	8	9	3
100	7	8649	93	10	8100	90
7	6	1	1	10	9	3
Сумма		MSE = 1236	MAE = 14.14		MSE = 1164	MAE = 15.43

Таблица 1: Модель a_1 игнорирует выброс (лучше по MAE). Модель a_2 подгоняется под выброс (лучше по MSE).

1.3 Гибридные функции: Huber Loss и Log-Cosh

Объединяют лучшие свойства MAE (устойчивость к выбросам) и MSE (гладкость в нуле).

Huber Loss:

$$L_{\delta}(y, a) = \begin{cases} \frac{1}{2}(y - a)^2, & |y - a| < \delta \\ \delta \left(|y - a| - \frac{1}{2}\delta \right), & |y - a| \geq \delta \end{cases} \quad \text{При } \delta \rightarrow 0 \Rightarrow \text{MAE}, \delta \rightarrow \infty \Rightarrow \text{MSE}.$$

- **Проблема:** В точках перехода ($|y - a| = \delta$) вторая производная имеет разрывы (изломы), что критично для алгоритмов оптимизации второго порядка. Параметр δ нужно подбирать вручную.

Log-Cosh:

$$L(y, a) = \log \cosh(a - y)$$

- **Применяется:** В продвинутых методах оптимизации. Она абсолютно гладкая во всех точках, бесконечно дифференцируема, не имеет разрывов. При малых ошибках ведет себя как парабола ($\approx \frac{x^2}{2}$), при больших — как прямая ($\approx |x| - \log(2)$).

1.4 MSLE (Mean Squared Logarithmic Error)

Формула:

$$L(y, a) = (\log(a + 1) - \log(y + 1))^2$$

- **Применяется:** Для задач с неотрицательными значениями, когда важен не абсолютный масштаб ошибки, а **отклонение в порядке величин** (например, ошибиться на 10 при прогнозе 20 — критично, а при прогнозе 10 000 — нет). Сильнее штрафует за занижение прогноза, чем за завышение.

1.5 Относительные метрики: MAPE и SMAPE

Используются, когда данные имеют принципиально разный масштаб (например, продажи штучных товаров и оптовых партий в одном датасете), чтобы крупные значения не доминировали при усреднении.

Формула MAPE:

$$\text{MAPE} = \frac{1}{\ell} \sum_{i=1}^{\ell} \left| \frac{y_i - a(x_i)}{y_i} \right|$$

- **Проблема:** Асимметрична. При $y = 1$ ошибка занижения ($a < y$) ограничена единицей, а ошибка завышения ($a > y$) сверху не ограничена.

Формула SMAPE (Симметричная модификация):

$$\text{SMAPE} = \frac{1}{\ell} \sum_{i=1}^{\ell} \frac{|y_i - a(x_i)|}{(|y_i| + |a(x_i)|)/2}$$

1.6 Квантильная функция потерь (Quantile Loss)

Формула:

$$Q(a, X) = \sum_{i=1}^{\ell} \rho_{\tau}(y_i - a(x_i)), \quad \text{где } \rho_{\tau}(z) = (\tau - 1)[z < 0]z + \tau[z \geq 0]z = (\tau - \frac{1}{2})z + \frac{1}{2}|z|$$

- **Применяется:** Когда асимметрична цена ошибки — стоимость недопрогноза и перепрогноза отличаются (например, в ритейле опаснее занизить спрос и потерять клиента, чем завысить и переплатить за склад). Параметр $\tau \in [0, 1]$ регулирует штраф за занижение. Оптимизация приводит к тому, что модель предсказывает τ -**квантиль** условного распределения ответов.

2 Градиентные методы оптимизации

2.1 Градиент и его свойства

Градиент функции $f: \mathbb{R}^d \rightarrow \mathbb{R}$ — вектор её частных производных: $\nabla f(x) = \left(\frac{\partial f}{\partial x_j} \right)_{j=1}^d$.

- **Свойство 1:** Градиент указывает направление наискорейшего **роста** функции, а антиградиент $(-\nabla f)$ — направление наискорейшего **убывания**.
- **Свойство 2:** Градиент всегда **ортогонален** линиям уровня функции потерь.

2.2 Классический градиентный спуск (Gradient Descent, GD)

Шаг итерации до сходимости ($Q(w)$ — ошибка на всей выборке):

$$w^{(k)} = w^{(k-1)} - \eta_k \nabla Q(w^{(k-1)})$$

- **Длина шага η_k :** При константном η есть риск «перешагнуть» минимум (если шаг велик) или сходиться слишком долго (если мал). Часто шаг уменьшают, например: $\eta_k = \frac{1}{k}$.
- **Критерии останова:** $\|\nabla Q(w)\| < \varepsilon$ (градиент близок к нулю) или $\|w^{(k)} - w^{(k-1)}\| < \varepsilon$ (веса перестали меняться).
- **Сходимость:** Для выпуклых гладких функций с условием Липшица сходится к минимуму со скоростью $Q(w^{(k)}) - Q(w^*) = O(1/k)$. На невыпуклых функциях может застрять в локальном минимуме или седловой точке.
- **Минус:** Слишком дорого вычислять полный градиент на каждом шаге по выборке размера ℓ .

2.3 Оценивание градиента на больших данных

1. Стохастический градиентный спуск (SGD): Полный градиент оценивается градиентом **одного** случайного объекта i_k :

$$w^{(k)} = w^{(k-1)} - \eta_k \nabla q_{i_k}(w^{(k-1)})$$

- **Плюсы:** Итерация мгновенная. Требуется в памяти всего 1 объект, что позволяет обучаться на гигантских выборках потоково (считывая по одному объекту с диска).
- **Минусы:** Теряет свойство затухания шага около минимума, траектория «шумная», метод может отдаляться от оптимума. Обязательно требует затухающего шага (условия Роббинса-Монро: $\sum \eta_k = \infty, \sum \eta_k^2 < \infty$). Скорость сходимости падает до $O(1/\sqrt{k})$.

2. Mini-batch Gradient Descent: Компромисс — градиент считается по **пачке (батчу)** из n случайных объектов.

3. Метод SAG (Stochastic Average Gradient): Хранит в памяти последние вычисленные градиенты для каждого объекта $z_i = \nabla q_i(w)$. На каждом шаге пересчитывается градиент только для *одного* случайного объекта i_k , а остальные берутся старыми из памяти:

$$w^{(k)} = w^{(k-1)} - \eta_k \frac{1}{\ell} \sum_{i=1}^{\ell} z_i^{(k)}$$

- **Плюс:** Сочетает скорость итерации SGD и крутую сходимость полного GD ($O(1/k)$).
- **Минус:** Требуется много памяти для хранения градиентов всех объектов (для линейных моделей оптимизируется хранением лишь скаляров — производных q'_i).

2.4 Модификации градиентного спуска

• **Momentum (Метод инерции):** Борется с осцилляциями (когда градиент прыгает туда-сюда на вытянутых линиях уровня), усредняя антиградиенты с прошлых шагов:

$$h_k = \alpha h_{k-1} + \eta_k \nabla_w Q(w^{(k-1)}), \quad w^{(k)} = w^{(k-1)} - h_k$$

• **AdaGrad:** Подбирает **свой размер шага для каждой компоненты веса**. Шаг тем меньше, чем длиннее были прошлые шаги по этой координате. *Идеален для разреженных задач* (где много редких признаков). *Минус:* Накопленный квадрат градиента G монотонно растет, шаги затухают до нуля еще до достижения минимума.

• **RMSprop:** Исправляет AdaGrad, вводя экспоненциальное затухание истории градиентов:

$$G_{kj} = \alpha G_{k-1,j} + (1 - \alpha) (\nabla_w Q(w^{(k-1)}))_j^2$$

Размер шага зависит только от интенсивности движения на *последних* итерациях.

- **Adam:** Объединяет идеи **Momentum** (накопление инерции для сглаживания шума) и **RMSprop** (адаптивный шаг по каждой координате). Самый популярный базовый оптимизатор.

3 Линейная регрессия, переобучение и валидация

3.1 Обучение линейной регрессии (Метод наименьших квадратов)

Задача оптимизации в матричном виде (X — матрица «объекты-признаки», y — вектор ответов):

$$Q(w) = \frac{1}{\ell} \|Xw - y\|^2 \rightarrow \min_w$$

Аналитическое решение (Явная формула):

$$w^* = (X^T X)^{-1} X^T y$$

- **Ограничения формулы:**

1. Вычислительная сложность обращения матрицы — $O(d^3)$, где d — число признаков. Неприменимо при тысячах признаков (требуются численные методы).
2. Матрица $X^T X$ может быть вырожденной или плохо обусловленной (мультиколлинеарность). Решается с помощью регуляризации.

3.2 Переобучение и Регуляризация

Переобучение (Overfitting) — ситуация, когда модель отлично подстраивается под обучающую выборку, но показывает плохое качество на новых данных (теряет обобщающую способность).

- **Симптом в линейных моделях:** Коэффициенты весов w становятся экстремально большими по модулю.
- **Регуляризация (Regularization):** Добавление в функционал ошибки штрафа за сложность модели (за норму вектора весов $\|w\|$), удерживающее коэффициенты от излишнего роста.

3.3 Оценивание качества моделей

1. **Отложенная выборка (Hold-out):** Данные делятся на обучающую (для настройки параметров) и контрольную (для проверки обобщающей способности).

Проблема: Результат сильно зависит от конкретного случайного разбиения.

2. **Кросс-валидация (Cross-Validation):** Данные бьются на k блоков примерно равного размера. Обучается k моделей, каждая на $k - 1$ блоках, а тестируется на оставшемся одном. Результаты усредняются:

$$CV = \frac{1}{k} \sum_{i=1}^k Q(a_i(x), X_i)$$

3. Построение финальной модели после CV:

- **Вариант А:** Обучить итоговую модель тем же способом на **всех** доступных данных (параметры будут точнее).
- **Вариант Б:** Если переобучать долго, построить **композицию** из полученных k моделей (например, усреднить их прогнозы в регрессии).

4 Линейная классификация и метрики качества

4.1 Линейный классификатор

Суть: Алгоритм строит разделяющую гиперплоскость и присваивает объекту класс на основе знака линейной комбинации признаков с весами. **Формула:**

$$a(x) = \text{sign}(\langle w, x \rangle + b) = \text{sign} \left(\sum_{j=1}^d w_j x_j + b \right)$$

Зачем надо: Базовый, самый быстрый и легко интерпретируемый алгоритм бинарной классификации (классы -1 и $+1$). Значение веса w_j напрямую отражает степень важности признака x_j .

4.2 Отступ (Margin)

Суть: Числовая величина, показывающая корректность и степень уверенности предсказания алгоритма на конкретном объекте (x_i, y_i) . **Формула:**

$$M_i = y_i(\langle w, x_i \rangle + b)$$

Анализ значений:

- $M_i > 0$ — Алгоритм классифицировал объект **верно**. Чем больше отступ, тем дальше точка находится в тылу своего класса.
- $M_i < 0$ — Алгоритм совершил **ошибку**. Объект попал на чужую сторону разделяющей плоскости.

Зачем надо: Прямой функционал ошибки (доля неправильных ответов) дискретен и имеет нулевой градиент, из-за чего модель невозможно обучать градиентным спуском. Отступ позволяет заменить реальный шаг потерь гладкими верхними оценками $\tilde{L}(M_i)$.

4.3 Метрики качества классификации

1. Доля правильных ответов (Accuracy)

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- **Критический минус:** Абсолютно неприменима при дисбалансе классов. Если в выборке 99% нормы и 1% мошенничества, тривиальный константный прогноз выдаст Accuracy = 99%, но модель будет бесполезна для бизнеса.

2. Точность (Precision) и Полнота (Recall)

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **Precision (Точность):** Доля истинных объектов среди всех, кого модель назвала положительными. Применяется, когда велика цена ложной тревоги ($\text{FP} \rightarrow 0$), например, блокировка легитимного клиента банком.
- **Recall (Полнота):** Доля объектов, которую модель смогла отыскать среди всех реально положительных объектов выборки. Применяется, когда критически важно не пропустить объект ($\text{FN} \rightarrow 0$), например, диагностика опасных заболеваний.

3. F_1 -мера (F_1 -score)

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Зачем надо: Гармоническое среднее точности и полноты. Если хотя бы один из параметров стремится к нулю, итоговая F -мера также падает, защищая от «слепых» перекошенных моделей.

4. Интегральные метрики: ROC-AUC и PR-AUC

- **ROC-AUC:** Площадь под кривой зависимости TPR от FPR. Показывает качество ранжирования вероятностей. Не зависит от баланса классов. У случайной модели $\text{AUC} = 0.5$, у идеальной — 1.0.
- **PR-AUC:** Площадь под кривой зависимости Precision от Recall. **Зачем надо:** В отличие от ROC-AUC, идеально работает в условиях жесткого дисбаланса классов, так как фокусируется строго на качестве прогноза дефицитного (положительного) класса.

5 Логистическая регрессия и Метод опорных векторов

5.1 Логистическая регрессия (Logistic Regression)

Суть: Алгоритм бинарной линейной классификации, который использует логистическую функцию потерь для выдачи непрерывной оценки вероятности принадлежности объекта к классу +1. **Формулы:**

- Логистическая функция потерь (Log-Loss):

$$L(y, z) = \log(1 + \exp(-yz))$$

- Сигмоидальная функция (сжимает диапазон $[-\infty; +\infty]$ в отрезок $[0; 1]$):

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

- Итоговый вероятностный прогноз модели:

$$P(y = 1 | x) = \sigma(\langle w, x \rangle + b)$$

Зачем надо: Ключевой инструмент в бизнес-задачах, где дискретного ответа «да/нет» недостаточно. Знание точной вероятности (например, 85% вероятности клика или дефолта) позволяет рассчитывать математическое ожидание прибыли и управлять рисками.

5.2 Метод опорных векторов (Support Vector Machine — SVM)

Суть: Геометрический метод классификации, максимизирующий ширину разделяющего коридора (зазора) между классами. Оптимальная прямая проходит строго посередине между ближайшими точками классов. **Формулы:**

- Функция потерь Hinge Loss (кусочно-линейный штраф):

$$L(y, z) = \max(0, 1 - yz) = (1 - yz)_+$$

- Задача безусловной оптимизации (Soft Margin SVM):

$$\frac{1}{2} \|w\|^2 + C \sum_{i=1}^{\ell} \max(0, 1 - y_i(\langle w, x_i \rangle + b)) \rightarrow \min_{w, b}$$

Ключевые понятия:

- **Опорные векторы** — Объекты обучающей выборки, которые попали точно на границу разделяющего зазора или нарушили её ($M_i \leq 1$). Положение разделяющей гиперплоскости зависит *только* от этих точек. Если убрать остальные («безопасные») объекты, плоскость не сдвинется.
- **Параметр регуляризации C** — Управляет балансом:
 - *Большой C* — Жесткий зазор (Hard Margin). Модель сильно штрафует за ошибки, зазор сужается, растёт риск переобучения.
 - *Маленький C* — Мягкий зазор (Soft Margin). Широкая полоса, алгоритм игнорирует отдельные выбросы ради достижения высокой обобщающей способности.

Зачем надо: Дает стабильное, математически оптимальное разделение выборки на геометрии данных средней размерности, минимизируя подгонку под шум.

6 Многоклассовая классификация

6.1 Сведение к серии бинарных задач

Если целевое множество содержит K классов ($\mathbb{Y} = \{1, \dots, K\}$), задачу можно решить декомпозицией на бинарные подзадачи [?].

1. Один против всех (One-versus-All / OvA) **Суть:** Обучается K независимых бинарных моделей $b_1(x), \dots, b_K(x)$, каждая из которых учится отличать свой k -й класс от всех остальных классов разом [?, ?].

- **Обучающая выборка для k -й модели:** $(x_i, 2[y_i = k] - 1)_{i=1}^{\ell}$ [?].
- **Правило предсказания:** Выбирается класс с максимальным уровнем уверенности:

$$a(x) = \arg \max_{k \in \{1, \dots, K\}} b_k(x)$$

- **Минус:** Масштабы выходов моделей могут кардинально различаться (несоизмеримость шкал), из-за чего прямое сравнение через $\arg \max$ математически некорректно [?]. Нормировка весов (например, в SVM) ломает оптимизационные свойства алгоритма [?, ?].

2. Все против всех (All-versus-All / AvA) **Суть:** Обучается $C_K^2 = \frac{K(K-1)}{2}$ бинарных классификаторов $a_{ij}(x)$ для каждой уникальной пары классов (i, j) [?].

- **Выборка для пары (i, j) :** Подвыборка $X_{ij} \subset X$, содержащая объекты исключительно классов i и j [?].
- **Правило предсказания:** Каждый классификатор отдает голос за один из двух классов, итоговый вердикт выносится мажоритарным голосованием:

$$a(x) = \arg \max_{k \in \{1, \dots, K\}} \sum_{i=1}^K \sum_{j \neq i} [a_{ij}(x) = k]$$

- **Минус:** При росте количества классов K число моделей растет квадратично, что делает подход вычислительно неприменимым на больших пулах категорий [?].

6.2 Многоклассовая логистическая регрессия

Суть: Прямое обобщение бинарной логистической регрессии на K классов с использованием оператора SoftMax, преобразующего вектор сырых линейных ответов в вероятностное распределение [?, ?]. **Формула вероятности для k -го класса:**

$$P(y = k | x, w) = \frac{\exp(\langle w_k, x \rangle + w_{0k})}{\sum_{j=1}^K \exp(\langle w_j, x \rangle + w_{0j})}$$

Обучение: Веса всех K моделей оптимизируются *одновременно* через максимизацию логарифма функции правдоподобия [?]:

$$\sum_{i=1}^{\ell} \log P(y = y_i | x_i, w) \rightarrow \max_{w_1, \dots, w_K}$$

Зачем надо: Позволяет строго и калиброванно оценивать многоклассовые вероятности без проблем со сходимостью и соизмеримостью шкал [?].

7 Классификация с пересекающимися классами (Multi-label)

Каждый объект x может обладать подмножеством из K независимых или скоррелированных меток одновременно: $y \in \{0, 1\}^K$ [?, ?].

7.1 Независимая классификация (Binary Relevance)

Суть: Задача разбивается на K абсолютно независимых бинарных задач [?]. Модель $b_k(x)$ обучается на векторе меток y_{ik} для k -го тега [?].

- **Проблема:** Метод полностью игнорирует внутренние зависимости, иерархические связи и корреляции между классами [?].

7.2 Стекинг классификаторов

Суть: Двухуровневый подход для неявного учета связей между метками классов [?].

1. Обучающая выборка разбивается на две непересекающиеся части X_1 и X_2 [?].
2. На выборке X_1 обучаются K базовых независимых моделей $b_1(x), \dots, b_K(x)$ [?].
3. Для объектов из отложенной выборки X_2 строятся новые мета-признаки, состоящие из прогнозов моделей первого уровня: $x'_{ik} = b_k(x_i)$ [?].
4. На мета-выборке X'_2 обучается второй ансамбль моделей $a_1(x'), \dots, a_K(x')$ [?]. Модели второго уровня видят предсказания коллег и корректируют итоговый вектор тегов [?].

Важное правило: Обучение моделей первого и второго уровня на одном и том же пуле данных X_1 запрещено. Это приводит к жесткой «утечке целевого признака» (target leak) и лавинообразному переобучению [?, ?].

7.3 Хэммингово расстояние (Метрика ошибки)

Оценивает долю неверно угаданных фактов принадлежности/непринадлежности к классам [?]:

$$\text{Hamming}(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} \frac{|Y_i \setminus Z_i| + |Z_i \setminus Y_i|}{K}$$

где Y_i — истинный набор тегов, Z_i — предсказанный набор тегов [?]. Метрика подлежит минимизации [?].

8 Обработка категориальных признаков

Пусть категориальный признак $f(x)$ принимает n уникальных строковых значений из множества $U = \{u_1, \dots, u_n\}$ [?, ?]. Линейные модели требуют обязательного перевода этих данных в числовой вид [?, ?].

8.1 Бинарное кодирование (One-Hot Encoding / ONE)

Суть: Исходный признак расщепляется на n независимых бинарных признаков-индикаторов [?]:

$$g_j(x) = [f(x) = u_j], \quad j = 1, \dots, n$$

- **Применяется:** Когда мощность множества n (число уникальных категорий) невелика. Линейные модели отлично работают с ONE из-за простоты оптимизации весов [?].
- **Не применяется:** При высокой мощности признака ($n \rightarrow 10^6$). Приводит к взрывному росту размерности матрицы признаков и дефициту оперативной памяти [?].

8.2 Бинарное кодирование с хэшированием (Hashing Trick)

Суть: Модификация ONE, где индекс целевого бинарного столбца вычисляется напрямую через детерминированную хэш-функцию $h : U \rightarrow \{1, \dots, B\}$, сжимающую пространство в фиксированный бюджет из B признаков [?, ?]:

$$g_j(x) = [h(f(x)) = j], \quad j = 1, \dots, B$$

- **Плюсы:** Не нужно хранить в памяти огромные словари соответствия строк индексам; фиксированный контролируемый расход памяти; редкие категории безопасно коллизируют, не ломая общую точность модели [?, ?, ?].

8.3 Счётчики (Target Encoding)

Суть: Замена строкового значения признака на K вещественных чисел, каждое из которых является сглаженной оценкой условной вероятности встретить класс k при условии данного значения признака: $g_k(x) \approx P(y = k \mid f(x))$ [?, ?]. **Формула вычисления признака:**

$$g_k(x, X) = \frac{\text{successes}_k(f(x), X) + c_k}{\text{counts}(f(x), X) + \sum_{m=1}^K c_m}$$

где $\text{counts}(u, X)$ — сколько раз категория встретилась в выборке, а successes_k — сколько раз она привела к классу k [?]. Константы c_k защищают от деления на ноль [?].

- **Защита от переобучения:** Чтобы избежать утечки ответов через счетчик, вычисление признаков для обучения проводят по схеме Out-of-Fold (аналог кросс-валидации): для блока X_i статистики собираются строго по оставшимся блокам $X \setminus X_i$ [?, ?]. На тесте используются глобальные счетчики всей обучающей выборки [?].

9 Решающие деревья: Базовые понятия

9.1 Математическая модель

Решающее дерево выполняет рекурсивное разбиение признакового пространства на J непересекающихся областей (предикатов) $R_1, \dots, R_J$ и возвращает константный прогноз w_j для каждой области. **Аналитическая форма записи:**

$$a(x) = \sum_{j=1}^J w_j [x \in R_j]$$

Зачем надо: Позволяет автоматически извлекать сложные нелинейные зависимости любой структуры без ручной генерации полиномиальных признаков. Модель полностью инвариантна к монотонным преобразованиям признаков (например, взятию логарифма).

9.2 Жадное построение (ID3 / C4.5 / CART)

Поиск глобально оптимальной структуры дерева — NP-полная задача. На практике применяется **жадный алгоритм** поочередного разделения узлов.

Выбор оптимального сплита (предиката): В каждом узле ищется пара «признак j — порог t », максимизирующая критерий информативности (Gain):

$$Q(X, j, t) = H(X) - \frac{|X_L|}{|X|} H(X_L) - \frac{|X_R|}{|X|} H(X_R) \rightarrow \max_{j, t}$$

где X — выборка в текущем узле, $X_L = \{x \in X \mid x^j < t\}$, $X_R = \{x \in X \mid x^j \geq t\}$, а $H(X)$ — функция меры хаоса (критерий информативности).

9.3 Критерии останова (Регуляризация)

Чтобы дерево не разрасталось до бесконечности, вводят жесткие ограничения сверху:

- Максимальная глубина дерева (`max_depth`).
- Минимальное число объектов в листе (`min_samples_leaf`), необходимое для статистической стабильности константы w_j .
- Ограничение на максимальное количество листьев (`max_leaf_nodes`).

10 Критерии информативности $H(X)$

Выбор функции $H(X)$ зависит сугубо от типа решаемой задачи машинного обучения. Пусть p_k — доля объектов класса k в текущем узле выборки X .

10.1 Задачи классификации

1. Миссклассификация (Ошибка классификации)

$$H(X) = 1 - \max_{k \in \{1, \dots, K\}} p_k$$

- **Минус:** Обладает плохими оптимизационными свойствами. Слабо чувствительна к изменению распределения долей классов на глубоких уровнях ветвления.

2. Критерий Джини (Неопределенность Джини) Характеризует вероятность перепутать класс объекта, если предсказывать его наугад в соответствии с распределением долей в узле:

$$H(X) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

Зачем надо: Базовый критерий для алгоритма CART. Стремится выделять крупные чистые («чистокровные») кластеры объектов.

3. Энтропийный критерий (Шенноновская энтропия)

$$H(X) = - \sum_{k=1}^K p_k \log_2 p_k$$

Зачем надо: Базовый критерий для алгоритмов ID3 и C4.5. Штрафует за хаос сильнее, чем критерий Джини, за счет логарифмической крутизны вблизи экстремумов.

10.2 Задачи регрессии

Для предсказания непрерывных величин в качестве хаоса выступает среднеквадратичное отклонение от среднего значения ответов узла $\bar{y}$:

$$H(X) = \frac{1}{|X|} \sum_{i \in X} (y_i - \bar{y})^2, \quad \text{где } \bar{y} = \frac{1}{|X|} \sum_{i \in X} y_i$$

Критерий минимизирует внутриузловую дисперсию целевой переменной.

11 Продвинутые аспекты решающих деревьев

11.1 Стрижка деревьев (Pruning)

Вместо жесткого останова построения эффективнее построить максимально глубокое переобученное дерево T_0 , а затем провести его «стрижку», минимизируя комбинированный функционал ошибок и сложности (**Cost-Complexity Pruning**):

$$R_\alpha(T) = R(T) + \alpha|T| \rightarrow \min_{T \subset T_0}$$

где $R(T)$ — суммарная ошибка дерева на обучении, $|T|$ — количество листьев, а α — гиперпараметр штрафа за сложность. Оптимальный α подбирается на кросс-валидации.

11.2 Обработка пропущенных значений (Missing Values)

Решающие деревья — один из немногих методов, способных обрабатывать пропуски в данных «из коробки»:

- **Суррогатные предикаты (CART):** Если для объекта x пропущен оптимальный признак j , узел рассчитывает «запасной» (суррогатный) признак j' , разбиение по которому максимально сильно коррелирует с оригинальным сплитом.
- **Распределение по весам (C4.5):** Объект с пропущенным признаком отправляется одновременно и в левое, и в правое поддерево, но с дробными весами $w_L = \frac{|X_L|}{|X|}$ и $w_R = \frac{|X_R|}{|X|}$.

11.3 Связь решающих деревьев с линейными моделями

Аналитическую запись дерева можно представить как линейную модель:

$$a(x) = \sum_{j=1}^J w_j \cdot \phi_j(x), \quad \text{где } \phi_j(x) = [x \in R_j]$$

Важный вывод: Жадный алгоритм дерева можно концептуально интерпретировать как автоматический нелинейный генератор новых бинарных признаков $\phi_j(x)$, над которыми в самом конце просто строится классическая линейная комбинация с весами w_j .

12 Разложение ошибки на смещение и разброс

12.1 Теоретическое обоснование

Для среднеквадратичной функции потерь $L(y, a) = (y - a(x))^2$ ожидаемая ошибка (полный риск) любого метода обучения $\mu(X)$, обученного по случайным выборкам $X \sim p(x, y)^\ell$, строго декомпозируется на три ортогональные компоненты [?, ?, ?, ?]:

$$L(\mu) = \text{Шум} + \text{Смещение}^2 + \text{Разброс} \quad (1)$$

12.2 Математические компоненты разложения

1. **Шум (Noise):** Ошибка идеального алгоритма $a_*(x) = \mathbb{E}[y | x]$, обусловленная неустранимой стохастической природой данных [?, ?, ?]:

$$\text{Шум} = \mathbb{E}_{x,y} [(y - \mathbb{E}[y | x])^2]$$

2. **Смещение (Bias):** Систематическая ошибка метода. Показывает отклонение усредненного по всем выборкам ответа алгоритма от истинного матожидания [?, ?]:

$$\text{Смещение} = \mathbb{E}_x [(\mathbb{E}_X [\mu(X)] - \mathbb{E}[y | x])^2]$$

3. **Разброс (Variance):** Дисперсия (чувствительность) алгоритма к малейшим флуктуациям и шумам в обучающей выборке X [?, ?, ?]:

$$\text{Разброс} = \mathbb{E}_x [\mathbb{E}_X [(\mu(X) - \mathbb{E}_X [\mu(X)])^2]]$$

12.3 Trade-off простых и сложных моделей

- **Простые модели** (например, линейные классификаторы) обладают *высоким смещением* (недообучение), но *низким разбросом* (устойчивы к замене точек выборки) [?, ?].
- **Сложные модели** (например, глубокие решающие деревья) обладают *низким смещением* (способны идеально интерполировать выборку), но *высоким разбросом* (склонны к жесткому переобучению) [?, ?].

13 Бэггинг (Bootstrap Aggregation)

13.1 Бутстрап-выборки

Пусть задана исходная обучающая выборка X размера ℓ [?]. **Бутстрап** — процедура генерации N новых выборок X_n путем равномерного извлечения ℓ объектов из X с возвращением [?, ?]. Вероятность того, что фиксированный объект выборки *не* попадет в бутстрап-выборку при $\ell \rightarrow \infty$ [?]:

$$P \approx \left(1 - \frac{1}{\ell}\right)^\ell \rightarrow \frac{1}{e} \approx 36.8\%$$

Зачем надо: Дублирование объектов эквивалентно неявному увеличению веса штрафа за ошибку на них, что обеспечивает различие базовых моделей [?].

13.2 Построение композиции бэггинга

Итоговый ансамбль строится как простое арифметическое усреднение независимых ответов N базовых алгоритмов $b_n(x)$ [?, ?, ?]:

$$a_N(x) = \frac{1}{N} \sum_{n=1}^N b_n(x)$$

13.3 Влияние бэггинга на Смещение и Разброс

- **Смещение:** Полностью сохраняется и совпадает со смещением одного базового алгоритма [?, ?]. Бэггинг не ухудшает выразительную способность ансамбля [?].
- **Разброс:** Зависит от степени скоррелированности базовых моделей. Если базовые модели полностью некоррелированы, то разброс композиции падает ровно в N раз [?]. При наличии корреляции (ρ) итоговый разброс ансамбля зажат ковариационной составляющей [?, ?]:

$$\text{Variance}(a_N) = \frac{1}{N} \sigma^2 + \frac{N-1}{N} \rho \sigma^2$$

Вывод: Бэггинг эффективен тогда и только тогда, когда в качестве базовых моделей берутся сложные алгоритмы с низким смещением и высоким разбросом (глубокие решающие деревья) [?, ?].

14 Случайный лес (Random Forest)

14.1 Алгоритм построения

Для минимизации корреляции ρ между деревьями вводится случайность по двум направлениям (объектам и признакам) [?, ?, ?]:

1. Для каждого $n = 1, \dots, N$ генерируется бутстрап-выборка $\tilde{X}_n$ [?, ?].
2. Строится глубокое решающее дерево $b_n(x)$ до тех пор, пока в листах не окажется $\leq n_{\min}$ объектов [?, ?].
3. **Метод случайных подпространств:** В каждом узле каждого дерева оптимальный сплит ищется *не* среди всего пространства признаков d , а среди случайного подмножества размера m [?, ?].

14.2 Рекомендуемые гиперпараметры (CART/Breiman)

Задача	Размер подмножества признаков m	Останов в листе $n_{\min}$
Классификация	$m = \lfloor \sqrt{d} \rfloor$ [?]	$n_{\min} = 1$ объект [?]
Регрессия	$m = \lfloor d/3 \rfloor$ [?]	$n_{\min} = 5$ объектов [?]

14.3 Out-of-Bag (OOB) Валидация

Поскольку каждое дерево обучается лишь на $\approx 63.2\%$ объектов, оставшиеся объекты являются для него чистым валидационным тестом [?]. **Формула OOB-ошибки ансамбля** [?]:

$$\text{OOB} = \sum_{i=1}^{\ell} L \left(y_i, \frac{1}{\sum_{n=1}^N [x_i \notin X_n]} \sum_{n=1}^N [x_i \notin X_n] b_n(x_i) \right)$$

Зачем надо: По мере роста N оценка сходится к честной кросс-валидации Leave-One-Out, позволяя контролировать переобучение абсолютно бесплатно, без затрат времени на повторное переобучение моделей [?].

14.4 Геометрическая интерпретация

Ответ Случайного леса аналитически сводится к взвешенной сумме ответов обучающей выборки [?, ?]:

$$a_N(x) = \sum_{i=1}^{\ell} \left(\frac{1}{N} \sum_{n=1}^N w_n(x, x_i) \right) y_i$$

где вес $w_n(x, x_i)$ пропорционален частоте попадания объектов x и x_i в один лист [?, ?]. Таким образом, Случайный лес неявно определяет адаптивную метрику близости на объектах и эквивалентен нелинейным метрическим методам [?, ?, ?]. Номера листьев $T_n(x)$ могут напрямую использоваться как мощные нелинейные признаки для моделей верхнего уровня [?, ?].

15 Градиентный бустинг: Математические основы

15.1 Концепция последовательного обучения

В отличие от бэггинга, бустинг строит композицию базовых алгоритмов $b_n(x) \in \mathcal{X}$ последовательно, где каждая новая модель минимизирует неустраняемую ошибку предыдущих [?, ?]. Итоговый ансамбль представляет собой взвешенную сумму [?]:

$$a_N(x) = \sum_{n=0}^N \gamma_n b_n(x)$$

где $b_0(x)$ — стартовое тривиальное приближение (обычно константа: среднее по ответам в регрессии или наиболее мощный класс в классификации) [?, ?].

15.2 Псевдоградиентный спуск в пространстве функций

Пусть задана дифференцируемая функция потерь $L(y, z)$ [?]. Задача выбора следующего оптимального шага ансамбля на итерации N имеет вид [?]:

$$\sum_{i=1}^{\ell} L(y_i, a_{N-1}(x_i) + \gamma_N b_N(x_i)) \rightarrow \min_{b_N, \gamma_N}$$

Для поиска направления наискорейшего убывания функционала вычисляются значения антиградиента (сдвигов) по прогнозам модели в точках обучающей выборки [?, ?]:

$$s_i^{(N)} = - \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=a_{N-1}(x_i)}, \quad i = 1, \dots, \ell$$

15.3 Построение базового алгоритма

Поскольку вектор антиградиента $s^{(N)} = (s_1^{(N)}, \dots, s_\ell^{(N)})$ определен только на объектах обучающей выборки, базовый алгоритм $b_N(x)$ обучается аппроксимировать данное направление на всем пространстве объектов [?, ?]. Оптимизируется вспомогательный квадратичный функционал потерь (независимо от вида исходного функционала L) [?, ?]:

$$b_N(x) = \arg \min_{b \in \mathcal{X}} \sum_{i=1}^{\ell} \left(b(x_i) - s_i^{(N)} \right)^2$$

После аппроксимации вектора сдвигов и фиксации структуры $b_N(x)$ одномерной оптимизацией подбирается масштабный коэффициент (шаг) [?]:

$$\gamma_N = \arg \min_{\gamma \in \mathbb{R}} \sum_{i=1}^{\ell} L(y_i, a_{N-1}(x_i) + \gamma b_N(x_i))$$

16 Регуляризация и Функции потерь бустинга

16.1 Методы борьбы с переобучением

Градиентный бустинг обладает экстремально высокой скоростью подгонки под выборку и склонен к переобучению на шуме [?].

1. Сокращение шага (Shrinkage / Learning Rate) Вместо полного шага в направлении аппроксимированного вектора сдвигов делается укороченный шаг с параметром темпа обучения $\eta \in (0, 1]$ [?]:

$$a_N(x) = a_{N-1}(x) + \eta \gamma_N b_N(x)$$

Зачем надо: Искусственное уменьшение доверия к локальному вектору антиградиента. Эмпирически доказано: уменьшение η при соразмерном увеличении общего числа итераций N повышает качество обобщающей способности ансамбля на тесте [?].

2. Стохастический градиентный бустинг (SGB) Каждый базовый алгоритм $b_N(x)$ обучается не на полном датасете X , а на случайной подвыборке без возвращения $X^k \subset X$ (типичный размер $|X^k| = 0.5 \cdot |X|$) [?, ?].

- **Плюсы:** Дополнительное нивелирование шума в данных; кратное ускорение вычислений шага [?].

16.2 Стандартные спецификации антиградиента

- **MSE регрессия:** $L(y, z) = \frac{1}{2}(z - y)^2 \implies s_i^{(N)} = y_i - a_{N-1}(x_i)$ (классический остаток прогноза) [?, ?].
- **MAE регрессия:** $L(y, z) = |y - z| \implies s_i^{(N)} = -\text{sign}(a_{N-1}(x_i) - y_i)$ [?]. Метод становится устойчивым к жестким выбросам.
- **Логистические потери (Бинарная классификация $Y = \{-1, +1\}$):**

$$L(y, z) = \log(1 + \exp(-yz)) \implies s_i^{(N)} = \frac{y_i}{1 + \exp(y_i a_{N-1}(x_i))}$$

Величина $w_i^{(N)} = \exp(-y_i a_{N-1}(x_i))$ выступает в качестве динамического веса важности объекта: если отступ большой положительный, объект практически полностью исключается из оптимизации текущего шага [?, ?].

17 Градиентный бустинг над решающими деревьями (GBDT)

17.1 Особенности композиции над деревьями

Решающее дерево разбивает пространство объектов на J_N непересекающихся областей (листьев) $R_{N1}, \dots, R_{NJ_N}$, возвращая константу b_{Nj} в каждой области [?]:

$$b_N(x) = \sum_{j=1}^{J_N} b_{Nj}[x \in R_{Nj}]$$

В GBDT шаг обновления ансамбля можно оптимизировать глубже: вместо поиска единого коэффициента γ_N перед деревом, константы оптимизируются независимо под каждый конкретный лист [?, ?].

17.2 Разделение задачи на подзадачи в листьях

Поскольку области разбиения листьев R_{Nj} взаимно ортогональны и не пересекаются, исходный многомерный функционал оптимизации распадается на J_N независимых одномерных подзадач [?]:

$$\gamma_{Nj} = \arg \min_{\gamma} \sum_{x_i \in R_{Nj}} L(y_i, a_{N-1}(x_i) + \gamma), \quad j = 1, \dots, J_N$$

Итоговый шаг корректировки ансамбля преобразуется к виду [?]:

$$a_N(x) = a_{N-1}(x) + \sum_{j=1}^{J_N} \gamma_{Nj}[x \in R_{Nj}]$$

17.3 Аналитические решения для листьев

Для классических задач оптимизации коэффициенты сдвига в листах находятся мгновенно без градиентных итераций:

- **Квадратичная ошибка (MSE):** $\gamma_{Nj} = \frac{1}{|R_{Nj}|} \sum_{x_i \in R_{Nj}} (y_i - a_{N-1}(x_i))$ — среднее значение остатков в листе. Дополнительный перепоиск не требуется, так как CART-дерево по умолчанию выставляет среднее.
- **Абсолютная ошибка (MAE):** $\gamma_{Nj} = \text{median}_{x_i \in R_{Nj}} \{y_i - a_{N-1}(x_i)\}$ — медиана остатков объектов, попавших в лист. Корректирует структуру дерева, изначально построенного по MSE-предикатам знаков ошибок.

Зачем надо: Использование GBDT над деревьями небольшой глубины (обычно от 3 до 6 уровней) позволяет получать наивысшее качество аппроксимации табличных данных в индустрии, сочетая высокую скорость инференса и устойчивость к нелинейным границам [?].

18 Extreme Gradient Boosting (XGBoost)

18.1 Математическая суть и аппроксимация 2-го порядка

В отличие от классического градиентного бустинга, который использует только первую производную (градиент) функции потерь $L(y, z)$, в XGBoost применяется разложение Тейлора до второго члена с центром в текущем ответе композиции $a_{N-1}(x_i)$:

$$\sum_{i=1}^{\ell} L(y_i, a_{N-1}(x_i) + b(x_i)) \approx \sum_{i=1}^{\ell} \left(L(y_i, a_{N-1}(x_i)) - s_i b(x_i) + \frac{1}{2} h_i b^2(x_i) \right)$$

Где векторы первых и вторых производных (сдвигов) вычисляются как:

$$s_i = - \left. \frac{\partial L(y_i, z)}{\partial z} \right|_{z=a_{N-1}(x_i)} \quad \text{и} \quad h_i = \left. \frac{\partial^2 L(y_i, z)}{\partial z^2} \right|_{z=a_{N-1}(x_i)}$$

После отбрасывания констант минимизируется модифицированный функционал:

$$\sum_{i=1}^{\ell} \left(-s_i b(x_i) + \frac{1}{2} h_i b^2(x_i) \right) \rightarrow \min_b$$

- **Применяется:** Для достижения максимальной точности и быстрой сходимости, так как учет вторых производных (кривизны функции потерь) позволяет делать более точный шаг в функциональном пространстве.
- **Не применяется:** Обычный бустинг эквивалентен этому подходу только при условии $h_i = 1$ (что соответствует отбрасыванию информации о кривизне ради гипотетической устойчивости).

18.2 Регуляризация дерева в XGBoost

Сложность базового дерева $b(x) = \sum_{j=1}^J b_j[x \in R_j]$ штрафует за число листьев J , так и за L_2 -норму коэффициентов в листьях:

$$\sum_{i=1}^{\ell} \left(-s_i b(x_i) + \frac{1}{2} h_i b^2(x_i) \right) + \gamma J + \frac{\lambda}{2} \sum_{j=1}^J b_j^2 \rightarrow \min_b$$

Оптимальные коэффициенты в листьях b_j и итоговая ошибка дерева $H(b)$ вычисляются аналитически через агрегированные по листьям производные $S_j = \sum_{i \in R_j} s_i$ и $H_j = \sum_{i \in R_j} h_i$:

$$b_j^* = \frac{S_j}{H_j + \lambda} \implies H(b) = -\frac{1}{2} \sum_{j=1}^J \frac{S_j^2}{H_j + \lambda} + \gamma J$$

18.3 Построение дерева и критерии

Функционал $H(b)$ выступает как **критерий информативности** для поиска разбиения $[x_j < t]$ в узле R :

$$Q = H(R) - H(R_\ell) - H(R_r) \rightarrow \max$$

- **Критерий останова:** Вершина объявляется листом, если даже наилучшее разбиение приводит к отрицательному значению прироста качества ($Q < 0$).

19 Композиции: Стекинг и Блендинг

19.1 Стекинг (Stacking)

Суть метода заключается в том, что прогнозы N независимо обученных базовых алгоритмов $b_1(x), \dots, b_N(x)$ объявляются новыми признаками, поверх которых обучается мета-алгоритм $a(x)$.

Для предотвращения переобучения выборка разбивается на K блоков. Базовые алгоритмы $b_j^{-k}(x)$ обучаются на всех блоках, кроме k -го, а мета-алгоритм оптимизируется на «чистых» предсказаниях:

$$\sum_{k=1}^K \sum_{(x_i, y_i) \in X_k} L(y_i, a(b_1^{-k}(x_i), \dots, b_N^{-k}(x_i))) \rightarrow \min_a$$

- **Применяется:** В соревнованиях по анализу данных (Kaggle) для объединения разнородных моделей (деревья, линейные модели, нейросети).
- **Не применяется:** Обучение мета-алгоритма на той же выборке, что и базовые модели, недопустимо, так как мета-алгоритм просто выберет наиболее переобученную базовую модель.

19.2 Блендинг (Blending)

Частный, наиболее простой случай стекинга, в котором мета-алгоритм представляет собой линейную комбинацию:

$$a(x) = \sum_{n=1}^N w_n b_n(x)$$

- **Применяется:** Для быстрого объединения моделей. Часто используется даже без настройки весов ($w_1 = \dots = w_N = 1/N$), что всё равно повышает качество за счет усреднения.

19.3 Обработка категориальных и текстовых признаков через Стекинг

Стандартные методы (One-Hot Encoding, TF-IDF) раздувают размерность признаков, из-за чего градиентный бустинг (имеющий небольшую глубину базовых деревьев) работает неэффективно или медленно.

Решение через стекинг:

1. Текстовые и категориальные признаки подаются на вход базовым алгоритмам (например, линейным моделям или через вычисление целевых счетчиков / Target Encoding).
2. Каждая сложная группа признаков «схлопывается» базовым алгоритмом в одно число (мета-признак).
3. На полученных мета-признаках обучается итоговый мета-алгоритм (например, XGBoost).

20 Ядровые методы: Теория и Трюк

20.1 Переход в спрямляющее пространство

Если зависимость между признаками нелинейна, можно применить нелинейное отображение в пространство более высокой размерности $\phi(x) : \mathbb{R}^d \rightarrow \mathbb{R}^D$, где $D \gg d$. Линейная модель в новом пространстве имеет вид:

$$f(x) = \langle w, \phi(x) \rangle + b$$

При явном вычислении $\phi(x)$ возникает проблема «проклятия размерности» (экспоненциальный рост числа признаков и затрат памяти).

20.2 Ядровой трюк (Kernel Trick)

Большинство линейных алгоритмов (SVM, линейная регрессия) при обучении и предсказании зависят только от скалярных произведений объектов $\langle \phi(x_i), \phi(x_j) \rangle$. **Суть трюка:** Замена скалярного произведения в функции потерь на функцию ядра $K(x, z)$, которая вычисляет скалярное произведение в пространстве $\mathbb{R}^D$, работая с векторами из исходного пространства $\mathbb{R}^d$:

$$K(x, z) = \langle \phi(x), \phi(z) \rangle$$

20.3 Теорема Мерсера (Условие существования ядра)

Функция $K(x, z)$ может быть ядром (то есть для неё существует спрямляющее пространство и отображение ϕ) тогда и только тогда, когда она:

1. Симметрична: $K(x, z) = K(z, x)$.
2. Неотрицательно определена: для любой выборки $\{x_i\}_{i=1}^\ell$ и любых вещественных чисел $c_1, \dots, c_\ell$ выполнено:

$$\sum_{i=1}^{\ell} \sum_{j=1}^{\ell} c_i c_j K(x_i, x_j) \geq 0$$

Матрица $K_{ij} = K(x_i, x_j)$ называется **матрицей Грама** (или матрицей ядер). Она должна быть полуположительно определенной.

20.4 Конструктор ядер (Свойства замыкания)

Если $K_1(x, z)$ и $K_2(x, z)$ — валидные ядра, то ядрами также являются:

- $K(x, z) = c \cdot K_1(x, z)$, где $c > 0$
- $K(x, z) = K_1(x, z) + K_2(x, z)$
- $K(x, z) = K_1(x, z) \cdot K_2(x, z)$
- $K(x, z) = f(x)f(z) \cdot K_1(x, z)$, для любой функции $f(x)$
- $K(x, z) = \exp(K_1(x, z))$

20.5 Основные типы ядер

1. **Полиномиальное:** $K(x, z) = (\langle x, z \rangle + R)^p$. Моделирует полиномы степени p . Параметр R регулирует вес низших степеней.
2. **Гауссово (RBF):** $K(x, z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right)$. Переносит данные в **бесконечномерное** пространство.
 - **Регуляризация σ :** Чем меньше σ , тем больше вес мономов высоких степеней, модель становится сложнее $\rightarrow$ риск **переобучения**. При $\sigma \rightarrow 0$ матрица Грама стремится к единичной (каждый объект выделяется в свой класс).

21 Ядровой метод опорных векторов (Kernel SVM)

21.1 Двойственная задача линейного SVM

Исходная (прямая) задача оптимизации SVM для неразделимой выборки:

$$\frac{1}{2}\|w\|^2 + C \sum_{i=1}^{\ell} \xi_i \rightarrow \min_{w, b, \xi} \quad \text{при } y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Переход к функции Лагранжа и максимизация по двойственным переменным λ_i (при условиях $\sum_{i=1}^{\ell} \lambda_i y_i = 0$ и $0 \leq \lambda_i \leq C$) приводит к задаче Куна-Таккера:

$$\sum_{i=1}^{\ell} \lambda_i - \frac{1}{2} \sum_{i=1}^{\ell} \sum_{j=1}^{\ell} \lambda_i \lambda_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \rightarrow \max_{\lambda}$$

21.2 Применение ядрового трюка к SVM

Заменяя скалярное произведение объектов $\langle x_i, x_j \rangle$ на функцию ядра $K(x_i, x_j)$, получаем итоговый функционал обучения ядрового SVM:

$$\sum_{i=1}^{\ell} \lambda_i - \frac{1}{2} \sum_{i=1}^{\ell} \sum_{j=1}^{\ell} \lambda_i \lambda_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \rightarrow \max_{\lambda}$$

Ограничения на параметры: $0 \leq \lambda_i \leq C$ и $\sum_{i=1}^{\ell} \lambda_i y_i = 0$.

21.3 Классификация новых объектов

Оптимальный вектор весов выражается через опорные объекты как $w = \sum_{i=1}^{\ell} \lambda_i y_i \phi(x_i)$. Итоговое решающее правило для нового объекта x вычисляется без явного нахождения w :

$$a(x) = \text{sign} \left(\sum_{i=1}^{\ell} \lambda_i y_i K(x_i, x) + b \right)$$

- Объекты, для которых $\lambda_i = 0$, не влияют на результат (лежат вне разделяющей полосы).
- Объекты с $\lambda_i > 0$ являются **опорными**.

21.4 Плюсы и минусы ядрового SVM

- + Позволяет строить сложнейшие нелинейные границы в исходном пространстве.
- + Задача выпуклая — решение всегда существует и оно единственно (нет локальных минимумов).
- Сложность обучения составляет $O(\ell^2)$ или $O(\ell^3)$ из-за работы с матрицей Грама размера $\ell \times \ell$. Метод **не применим** на больших выборках ($\ell > 100\,000$).

21.5 Метод случайных признаков (Random Fourier Features)

Способ обойти вычислительную сложность $O(\ell^3)$ для больших выборок при использовании RBF-ядра. **Суть:** Вместо точного ядра генерируется конечное число n случайных признаков с помощью аппроксимации Монте-Карло:

$$\tilde{\phi}(x) = \frac{1}{\sqrt{n}} (\cos(w_1^T x), \sin(w_1^T x), \dots, \cos(w_n^T x), \sin(w_n^T x))$$

Где векторы весов w_j сэмплируются случайным образом из нормального распределения $\mathcal{N}(0, \frac{1}{\sigma^2} I)$. Скалярное произведение $\langle \tilde{\phi}(x), \tilde{\phi}(z) \rangle$ является несмещенной оценкой гауссова ядра $K(x, z)$. Это позволяет свести задачу к обычному быстрому линейному SVM.

22 Смеси распределений и ЕМ-алгоритм

22.1 Смеси распределений

Плотность вероятности выборки $p(x)$ называется **смесью K распределений**, если она представима в виде:

$$p(x | \Theta) = \sum_{k=1}^K \pi_k p_k(x | \theta_k)$$

Где $\pi_k \geq 0$, $\sum_{k=1}^K \pi_k = 1$ — априорные вероятности (веса) компонентов смеси, а θ_k — параметры k -го распределения. Полный набор параметров: $\Theta = \{\pi_k, \theta_k\}_{k=1}^K$.

Для каждого объекта x_i вводится скрытая переменная $z_i \in \{1, \dots, K\}$, указывающая на номер компонента, породившего данный объект:

$$p(z_i = k | \Theta) = \pi_k \quad \text{и} \quad p(x_i | z_i = k, \Theta) = p_k(x_i | \theta_k)$$

22.2 Проблема неполных данных и сломанный ММП

Логарифм функции правдоподобия для независимой выборки $X = \{x_i\}_{i=1}^\ell$ имеет вид:

$$\log p(X | \Theta) = \sum_{i=1}^\ell \log \left(\sum_{k=1}^K \pi_k p_k(x_i | \theta_k) \right) \rightarrow \max_{\Theta}$$

Из-за суммы под знаком логарифма аналитическое взятие производных невозможно.

22.3 Нижняя вариационная оценка правдоподобия (ELBO)

Для обхода суммы вводится произвольное распределение скрытых переменных $q(Z)$. Логарифм правдоподобия раскладывается через Jensen's inequality и KL-дивергенцию:

$$\log p(X | \Theta) = \mathcal{P}(q, \Theta) + \text{KL}(q(Z) | p(Z | X, \Theta))$$

Где $\mathcal{P}(q, \Theta)$ — нижняя вариационная оценка (**ELBO**), максимизация которой гарантирует рост истинного правдоподобия:

$$\mathcal{P}(q, \Theta) = \sum_Z q(Z) \log \frac{p(X, Z | \Theta)}{q(Z)} = \mathbb{E}_q[\log p(X, Z | \Theta)] + H(q)$$

22.4 Итерации ЕМ-алгоритма

Алгоритм стартует с начального приближения Θ^0 и поочередно выполняет два шага:

Е-шаг (Expectation): Фиксируются параметры Θ^i . Ищется распределение $q(Z)$, максимизирующее $\mathcal{P}(q, \Theta^i)$. Максимум достигается при полном занулении KL-дивергенции, то есть когда $q(Z)$ совпадает с истинным апостериорным распределением:

$$q(z_i = k) = \gamma_{ik} = p(z_i = k | x_i, \Theta^i) = \frac{\pi_k^i p_k(x_i | \theta_k^i)}{\sum_{j=1}^K \pi_j^i p_j(x_i | \theta_j^i)}$$

Величина γ_{ik} — это «скрытая ответственность» (вероятность принадлежности объекта x_i к кластеру k).

М-шаг (Maximization): Фиксируются посчитанные вероятности γ_{ik} . Ищутся параметры Θ , максимизирующие ELBO:

$$\Theta^{i+1} = \arg \max_{\Theta} \sum_{i=1}^\ell \sum_{k=1}^K \gamma_{ik} \log (\pi_k p_k(x_i | \theta_k))$$

Для весов компонентов смеси π_k решение всегда находится аналитически через метод Лагранжа:

$$\pi_k^{i+1} = \frac{1}{\ell} \sum_{i=1}^\ell \gamma_{ik}$$

22.5 Гауссова смесь (GMM — Gaussian Mixture Model)

Если компоненты смеси — многомерные нормальные распределения $p_k(x | \theta_k) = \mathcal{N}(x | \mu_k, \Sigma_k)$, то на **М-шаге** формулы обновления параметров имеют вид:

$$\mu_k^{i+1} = \frac{\sum_{i=1}^\ell \gamma_{ik} x_i}{\sum_{i=1}^\ell \gamma_{ik}} \quad \text{и} \quad \Sigma_k^{i+1} = \frac{\sum_{i=1}^\ell \gamma_{ik} (x_i - \mu_k^{i+1})(x_i - \mu_k^{i+1})^T}{\sum_{i=1}^\ell \gamma_{ik}}$$

22.6 Свойства ЕМ-алгоритма

- + На каждой итерации значение целевого правдоподобия $\log p(X | \Theta)$ гарантированно не уменьшается.
- + Естественным образом решает задачу мягкой кластеризации (каждый объект принадлежит кластеру с определенной вероятностью).
- Алгоритм сходится лишь к **локальному минимуму/оптимуму** (критически важна хорошая начальная инициализация параметров, например, с помощью алгоритма *K*-means).

23 Несбалансированные выборки и Поиск аномалий

23.1 Методы сэмплирования выборки

В задачах с сильным дисбалансом классов (доля аномалий $\ll 1\%$) стандартные функционалы качества несостоятельны. Применяются подходы модификации выборки:

1. **Subsampling (Undersampling):** Случайное удаление объектов мажоритарного класса. Приводит к потере информации.
2. **Oversampling:** Дублирование объектов миноритарного класса. Ведет к переобучению.
3. **Алгоритм SMOTE (Synthetic Minority Over-sampling Technique):** Для каждого объекта x_i миноритарного класса вычисляются k ближайших соседей из того же класса. Случайно выбирается сосед x^{nn} . Новый синтетический объект $\tilde{x}$ генерируется на отрезке между ними:

$$\tilde{x} = x_i + \lambda(x^{nn} - x_i), \quad \lambda \sim U[0, 1]$$

23.2 Одноклассовая классификация (One-Class Classification)

Постановка задачи: задана выборка только «нормальных» объектов $X = \{x_i\}_{i=1}^{\ell}$. Требуется построить решающее правило $a(x) \in \{-1, 1\}$, разделяющее распределение нормальных данных (1) и область аномалий (-1).

23.3 One-Class SVM (OC-SVM)

Алгоритм Schölkopf. Идея состоит в отделении выборки от начала координат гиперплоскостью $\langle w, x \rangle - \rho = 0$ на максимальный отступ $\frac{\rho}{\|w\|}$ с допущением ошибок ξ_i :

$$\frac{1}{2}\|w\|^2 - \rho + \frac{1}{\nu\ell} \sum_{i=1}^{\ell} \xi_i \rightarrow \min_{w, \rho, \xi} \quad \text{при } \langle w, x_i \rangle \geq \rho - \xi_i, \quad \xi_i \geq 0$$

Параметр $\nu \in (0, 1]$ регулирует долю аномалий: он является верхней оценкой доли «ошибок» (выбросов выборки) и нижней оценкой доли опорных векторов.

При использовании ядрового трюка через RBF-ядро $K(x, z) = \exp(-\gamma\|x - z\|^2)$ двойственная задача принимает вид:

$$\frac{1}{2} \sum_{i=1}^{\ell} \sum_{j=1}^{\ell} \lambda_i \lambda_j K(x_i, x_j) \rightarrow \min_{\lambda} \quad \text{при } 0 \leq \lambda_i \leq \frac{1}{\nu\ell}, \quad \sum_{i=1}^{\ell} \lambda_i = 1$$

Итоговое решающее правило (значение функции определяет степень аномальности):

$$a(x) = \text{sign} \left(\sum_{i=1}^{\ell} \lambda_i K(x_i, x) - \rho \right)$$

23.4 Изоляционный лес (Isolation Forest)

Непараметрический метод, основанный на случайных решающих деревьях. Основан на свойстве: для изолирования аномальных (удаленных) объектов в пространстве требуется существенно меньше случайных разбиений, чем для объектов из областей высокой плотности.

Алгоритм построения дерева: В текущей вершине случайно выбирается признак j и случайный порог $t \in [\min X_{\cdot j}, \max X_{\cdot j}]$. Разбиение продолжается до полной изоляции каждого объекта, либо до достижения максимальной глубины.

Расчет меры аномальности: Пусть $h_n(x)$ — длина пути (число ребер) от корня до листа для объекта x в n -м дереве. Если дерево остановилось на листе размера m , то итоговая длина пути нормируется на среднюю длину пути в бинарном дереве поиска из m элементов:

$$h_n(x) = k + c(m), \quad \text{где } c(m) = 2H(m-1) - \frac{2(m-1)}{m}$$

Где $H(i) \approx \ln(i) + 0.5772$ — гармоническое число. Мера аномальности объекта по ансамблю из N деревьев вычисляется как:

$$a(x) = 2^{-\frac{\frac{1}{N} \sum_{n=1}^N h_n(x)}{c(\ell)}}$$

- Если $a(x) \rightarrow 1$ ($\sum h_n(x) \rightarrow 0$) — длина путей минимальна, объект строго **аномален**.

- Если $a(x) \rightarrow 0$ — объект лежит глубоко в структуре данных, **норма**.
- Обучение можно ускорить, строя деревья на случайных подвыборках фиксированного размера $s \ll \ell$ (тогда в знаменателе используется $c(s)$).

24 Графовая и спектральная кластеризация

24.1 Представление выборки в виде графа

Для кластеризации объектов сложной нелинейной формы выборка $X = \{x_i\}_{i=1}^\ell$ моделируется неориентированным графом $G = (V, E)$. Вершины V — объекты выборки. Матрица смежности (матрица весов / сходства) $W = (w_{ij})$ задает близость между объектами:

1. **ε -neighborhood graph:** $w_{ij} = 1$, если $\|x_i - x_j\| < \varepsilon$. Граф часто распадается на несвязные компоненты.
2. **k -nearest neighbor graph:** вершины i и j соединяются, если i входит в число k ближайших соседей j , либо наоборот. При взаимном условии получается *mutual k -NN graph*.
3. **Fully connected graph (с RBF-ядром):** все пары объектов соединяются ребром с весом:

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

Степень вершины определяется как суммарный вес её ребер: $d_i = \sum_{j=1}^\ell w_{ij}$. Матрица степеней $D = \text{diag}(d_1, \dots, d_\ell)$ — диагональная.

24.2 Критерии разреза графа

Разбиение графа на два кластера A и B ($B = V \setminus A$) оценивается весом разрезанных ребер — функционал **MinCut**:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$$

Минимизация $\text{cut}(A, B)$ приводит к выделению изолированных вершин-выбросов. Для балансировки размеров кластеров применяются нормированные критерии:

1. **RatioCut** (нормировка на число вершин):

$$\text{RatioCut}(A, B) = \frac{\text{cut}(A, B)}{|A|} + \frac{\text{cut}(A, B)}{|B|}$$

2. **NormalizedCut (NCut)** (нормировка на объем/сумму степеней вершин кластера):

$$\text{NCut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}, \quad \text{где } \text{vol}(A) = \sum_{i \in A} d_i$$

24.3 Матрица Лапласа графа (Laplacian)

Минимизация RatioCut и NCut в дискретном виде NP-трудна. Непрерывное расслабление задачи приводит к спектральному анализу Лапласианов:

1. **Ненормированный Лапласиан:** $L = D - W$.

- Свойства: L симметрична и полуположительно определена ($f^T L f = \frac{1}{2} \sum_{i,j} w_{ij} (f_i - f_j)^2 \geq 0$).
- Кратность собственного значения $\lambda = 0$ равна числу связных компонент графа, а соответствующие собственные векторы — индикаторные функции этих компонент.

2. **Нормированные Лапласианы:**

$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}$$

$$L_{\text{rw}} = D^{-1} L = I - D^{-1} W$$

24.4 Алгоритм спектральной кластеризации (по Ши и Малику)

Аппроксимирует минимизацию функционала NCut на K кластеров:

1. Построить матрицу сходства W и диагональную матрицу степеней D .
2. Вычислить ненормированный Лапласиан $L = D - W$.
3. Найти K наименьших собственных векторов $u_1, \dots, u_K$ обобщенной задачи на собственные значения $Lu = \lambda Du$ (что эквивалентно собственным векторам матрицы L_{rw}).
4. Сформировать матрицу $U \in \mathbb{R}^{\ell \times K}$, где столбцы — векторы u_j .
5. Рассматривать строки матрицы U как новые признаки объектов в пространстве $\mathbb{R}^K$. Кластеризовать их стандартным алгоритмом **K -means**.

24.5 Внешние метрики качества кластеризации

Пусть Y — истинные метки классов (gold standard), C — метки, выданные алгоритмом. Обозначим пары объектов: a — в одном классе и одном кластере; b — в одном классе, но разных кластерах; c — в разных классах, но одном кластере; d — в разных классах и разных кластерах.

1. **Rand Index (RI)**: доля пар объектов, классифицированных согласованно:

$$\text{RI} = \frac{a + d}{a + b + c + d}$$

2. **Adjusted Rand Index (ARI)**: скорректирован на случайный результат:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \in [-1, 1]$$

3. **BCubed**: Метрика считает точность и полноту индивидуально для каждого объекта x . Пусть $L(x)$ — истинный класс, $C(x)$ — полученный кластер:

$$\text{Correctness}(x, x') = \begin{cases} 1, & C(x) = C(x') \wedge L(x) = L(x') \\ 0, & \text{иначе} \end{cases}$$

$$\text{Precision-BCubed} = \text{Avg}_x \left[\text{Avg}_{x': C(x)=C(x')} \text{Correctness}(x, x') \right]$$

$$\text{Recall-BCubed} = \text{Avg}_x \left[\text{Avg}_{x': L(x)=L(x')} \text{Correctness}(x, x') \right]$$

Итоговое качество — стандартная F_1 -мера от полученных Precision и Recall.

25 Метрические методы классификации и LSH

25.1 Классификатор k ближайших соседей (kNN)

Задана выборка $X = (x_i, y_i)_{i=1}^\ell$, $y_i \in \mathbb{Y} = \{1, \dots, K\}$ и симметричная функция расстояния $\rho(x, z)$. Пусть $x_u^{(1)}, \dots, x_u^{(\ell)}$ — объекты обучающей выборки, отсортированные по неубыванию расстояния до нового объекта u :

$$\rho(u, x_u^{(1)}) \leq \rho(u, x_u^{(2)}) \leq \dots \leq \rho(u, x_u^{(\ell)})$$

Взвешенный kNN: Решающее правило сопоставляет объекту u вес за каждый класс, используя весовую функцию $w(i, \rho)$:

$$a(x) = \arg \max_{y \in \mathbb{Y}} \sum_{i=1}^k [y_u^{(i)} = y] \cdot w(i, \rho(u, x_u^{(i)}))$$

Варианты задания весов $w(i, d)$:

1. Обычный kNN: $w(i, d) = 1$.
2. Убывание от номера: $w(i, d) = q^i$ ($q < 1$) или $w(i, d) = \frac{1}{i}$.
3. Парзеновское окно (зависимость от расстояния): $w(i, d) = K\left(\frac{d}{h}\right)$, где $K(z)$ — функция ядра (например, Гауссово, Епанечникова), а h — ширина окна.

25.2 Проклятие размерности в метрических пространствах

С ростом размерности пространства $d \rightarrow \infty$ метрические свойства выборки вырождаются:

1. Расстояния между объектами стабилизируются. Отношение разности максимального и минимального расстояния к минимальному стремится к нулю:

$$\lim_{d \rightarrow \infty} \frac{\max_i \rho(u, x_i) - \min_i \rho(u, x_i)}{\min_i \rho(u, x_i)} = 0$$

Понятие «ближайшего соседа» теряет физический смысл, так как все объекты становятся равноудаленными.

2. Объем гиперсферы радиуса R сосредоточен в тонком приграничном слое. При случайном сэмпировании все точки оказываются на «границах» области.

25.3 Приближенный поиск соседей (ANN) и LSH

Для больших выборок точный поиск соседей за $O(\ell d)$ неэффективен. Применяется подход **Locality-Sensitive Hashing (LSH)**: разбиение пространства на бакеты с помощью семейства хэш-функций $\mathcal{H}$, сохраняющих близость объектов.

Семейство $\mathcal{H}$ называется (r_1, r_2, p_1, p_2) -чувствительным, если для любых $x, z \in \mathbb{X}$:

- Если $\rho(x, z) \leq r_1$, то $\mathcal{P}[h(x) = h(z)] \geq p_1$ (близкие попадают в один бакет).
- Если $\rho(x, z) \geq r_2$, то $\mathcal{P}[h(x) = h(z)] \leq p_2$ (далекие разделяются).

25.4 LSH для евклидова расстояния (Метод случайных проекций)

Для пространства $\mathbb{R}^d$ с L_2 -метрикой хэш-функция проецирует вектор на случайную прямую w с последующим квантованием:

$$h_{w,b}(x) = \left\lfloor \frac{\langle w, x \rangle + b}{W} \right\rfloor$$

Где параметры генерируются случайно:

- $w \in \mathbb{R}^d$ — вектор, компоненты которого сэмпированы независимо из стандартного нормального распределения: $w_j \sim \mathcal{N}(0, 1)$.
- b — случайный сдвиг, сэмплируемый из равномерного распределения: $b \sim U[0, W]$.
- W — фиксированная ширина шага (гиперпараметр бакета).

25.5 Увеличение точности: Конструкции AND и OR

Чтобы скорректировать вероятности p_1 и p_2 , базовые хэш-функции объединяют:

1. **AND-конструкция (уменьшает количество ложноположительных):** Объект хэшируется вектором из m независимых функций $g(x) = (h_1(x), \dots, h_m(x))$. Совпадение бакетов требует равенства всех компонентов:

$$p_{\text{AND}} = p^m$$

2. **OR-конструкция (увеличивает полноту поиска):** Строится L независимых таблиц, в каждой из которых используется своя AND-комбинация. Объект объявляется кандидатом в соседи, если он совпал хотя бы в одной таблице:

$$p_{\text{OR}} = 1 - (1 - p^m)^L$$

Подбором m и L достигается S-образная кривая вероятности совпадения, жестко отсекающая объекты на заданном расстоянии.